

Universidad de San Carlos de Guatemala
Facultad de Ingeniería — Ingeniería en Ciencias y Sistemas
Sistemas de Bases de Datos 2

MANUAL DE USUARIO

Sistema de Gestión de Reservas y Boletos Aéreos con Apache Cassandra

Proyecto 1 — Repositorio: https://github.com/Halex07/BD2_2S26_Proyectos_201407049.git

1. Introducción

Este manual explica cómo instalar, levantar y operar el clúster de Apache Cassandra del Sistema de Gestión de Reservas y Boletos Aéreos, cómo cargar los datos, cómo ejecutar las 5 consultas de negocio requeridas y cómo reproducir las pruebas de tolerancia a fallos y de Consistency Level (ONE, QUORUM, ALL).

El sistema modela una aerolínea (keyspace aerolinea) con las entidades Pasajeros, Aeronaves, Vuelos, Asientos, Reservas y Pagos, usando un diseño orientado a queries (Query-Driven Modeling): cada tabla especializada responde una consulta específica sin JOINS ni ALLOW FILTERING.

1.1 Alcance de este documento

- Requisitos e instalación del clúster de 3 nodos (Docker).
- Estructura del keyspace y las tablas Cassandra.
- Carga masiva de datos (Batch Writes) y reglas de negocio (LWT y TTL).
- Las 5 consultas CQL de negocio, con parámetros y resultados de ejemplo.
- Simulación de caída de nodos y pruebas de Consistency Level.
- Resolución de problemas comunes.

2. Requisitos previos

- Docker y Docker Compose (clúster local de Apache Cassandra 4.x, 3 nodos).
- Python 3.10+ con las librerías: cassandra-driver, faker, pandas.
- cqlsh (incluido en la imagen de Cassandra, se ejecuta dentro del contenedor).
- PowerShell o una terminal con acceso a Docker (Windows/Linux/Mac).

Instalación de dependencias de Python dentro del entorno del proyecto:

```
pip install cassandra-driver faker pandas
```

3. Instalación y despliegue del clúster

El clúster está compuesto por 3 nodos (cass1, cass2, cass3) desplegados con Docker, en el datacenter dc1, rack1, cada uno con 16 tokens y Owns (effective) de 100% una vez estabilizado el anillo.

3.1 Levantar el clúster

```
# Desde la carpeta del proyecto (donde está el docker-compose.yml)
docker compose up -d

# Verificar que los 3 nodos estén activos (UN = Up/Normal)
docker exec cass1 nodetool status
```

Salida esperada (los 3 nodos en estado UN):

```
Datacenter: dc1
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
-- Address      Load        Tokens      Owns (effective)  Host ID      Rack
UN  172.18.0.4    34.64 MiB    16          100.0%            ...          rack1
UN  172.18.0.3    34.63 MiB    16          100.0%            ...          rack1
UN  172.18.0.2    34.62 MiB    16          100.0%            ...          rack1
```

3.2 Detener y reiniciar nodos individuales

Para simular una caída de nodo se detiene el contenedor correspondiente; nodetool status lo mostrará como DN (Down/Normal) en los demás nodos:

```
docker stop cass2
docker exec cass1 nodetool status
# cass2 aparece como DN (Down)

# Reiniciar el/los nodo(s) detenidos
docker start cass2 cass3
docker exec cass1 nodetool status
# Todos vuelven a UN una vez sincronizados
```

```
try Docker debug for seamless, persistent debugging tools in any container or image → docker debug cass1
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker stop cass3
cass3
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker exec cass1 nodetool status
Datacenter: dc1
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
-- Address      Load          Tokens   Owns (effective)  Host ID                               Rack
DN  172.18.0.4   34.64 MiB     16       100.0%           14873b2e-ec80-46b2-9e5d-d7b731000845  rack1
UN  172.18.0.3   34.65 MiB     16       100.0%           9dbe660f-68b9-488c-84ce-f0525e13cfb5  rack1
UN  172.18.0.2   34.6 MiB      16       100.0%           69cb9021-8ddd-4c9b-8cf2-539168bcac19  rack1

PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> |
```

Figura 3.1 — docker stop cass3: el nodo cass3 pasa a DN (Down) en nodetool status.

```
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker stop cass2
cass2
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker exec cass1 nodetool status
Datacenter: dc1
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
-- Address      Load          Tokens   Owns (effective)  Host ID                               Rack
DN  172.18.0.4   34.64 MiB     16       100.0%           14873b2e-ec80-46b2-9e5d-d7b731000845  rack1
DN  172.18.0.3   34.65 MiB     16       100.0%           9dbe660f-68b9-488c-84ce-f0525e13cfb5  rack1
UN  172.18.0.2   34.6 MiB      16       100.0%           69cb9021-8ddd-4c9b-8cf2-539168bcac19  rack1

PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> |
```

Figura 3.2 — docker stop cass2: con cass2 y cass3 detenidos, solo cass1 permanece UN (Up/Normal).

```
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker start cass2 cass3
cass2
cass3
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker exec cass1 nodetool status
Datacenter: dc1
=====
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
-- Address      Load          Tokens   Owns (effective)  Host ID                               Rack
DN  172.18.0.4   34.64 MiB     16       100.0%           14873b2e-ec80-46b2-9e5d-d7b731000845  rack1
DN  172.18.0.3   34.65 MiB     16       100.0%           9dbe660f-68b9-488c-84ce-f0525e13cfb5  rack1
UN  172.18.0.2   34.6 MiB      16       100.0%           69cb9021-8ddd-4c9b-8cf2-539168bcac19  rack1

PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker exec cass1 nodetool repair aerolinea
```

Figura 3.3 — docker start cass2 cass3: los nodos vuelven a UN; a continuación se lanza nodetool repair aerolinea.

3.3 Reparar datos tras una caída

Después de reiniciar un nodo que estuvo caído, se recomienda ejecutar un repair para asegurar que los datos escritos durante la caída se propaguen a la réplica que estuvo abajo:

```
docker exec cass1 nodetool repair aerolinea
```

3.4 Verificar el keyspace y las tablas

```
docker exec cass1 nodetool status aerolinea
docker exec cass1 nodetool tablestats aerolinea.reservas_por_id
```

Este comando reporta, entre otros datos, el número de tablas del keyspace, el conteo de escrituras/lecturas, el tamaño en disco y la latencia local de lectura/escritura de la tabla — útil para verificar que la carga masiva se distribuyó correctamente entre los nodos.

```
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker exec cass1 nodetool status aerolinea
Datacenter: dc1
=====
Status=Up/Down
-- State=Normal/Leaving/Joining/Moving
-- Address      Load       Tokens     Owns (effective)  Host ID                               Rack
UN 172.18.0.4    34.64 MiB  16        100.0%            14873b2e-ec80-46b2-9e5d-d7b731000845 rack1
UN 172.18.0.3    34.63 MiB  16        100.0%            9dbe660f-68b9-488c-84ce-f0525e13cfb5 rack1
UN 172.18.0.2    34.62 MiB  16        100.0%            69cb9021-8ddd-4c9b-8cf2-539168bcac19 rack1

PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker exec cass1 nodetool tablestats aerolinea.reservas_por_id
Total number of tables: 56
=====
```

Figura 3.4 — Los 3 nodos en estado UN y arranque del comando tablestats sobre aerolinea.reservas_por_id (56 tablas en el keyspace).

```
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker exec cass1 nodetool tablestats aerolinea.reservas_por_id
Total number of tables: 56
-----
Keyspace : aerolinea
  Read Count: 14
  Read Latency: 0.6079285714285715 ms
  Write Count: 530052
  Write Latency: 0.03058236173054719 ms
  Pending Flushes: 0
    Table: reservas_por_id
      SSTable count: 3
      Old SSTable count: 0
      Space used (live): 8971021
      Space used (total): 8971021
      Space used by snapshots (total): 0
      Off heap memory used (total): 149864
      SSTable Compression Ratio: 0.5947624888399602
      Number of partitions (estimate): 99781
      Memtable cell count: 1106
      Memtable data size: 225245
      Memtable off heap memory used: 0
      Memtable switch count: 9
      Speculative retries: 0
      Local read count: 0
      Local read latency: NaN ms
      Local write count: 100000
      Local write latency: 0.022 ms
      Pending flushes: 0
      Percent repaired: 0.0
      Bytes repaired: 0.000KiB
      Bytes unrepaired: 10.510MiB
      Bytes pending repair: 0.000KiB
      Bloom filter false positives: 0
      Bloom filter false ratio: 0.00000
      Bloom filter space used: 122816
      Bloom filter off heap memory used: 122792
      Index summary off heap memory used: 21672
      Compression metadata off heap memory used: 5400
      Compacted partition minimum bytes: 104
      Compacted partition maximum bytes: 124
      Compacted partition mean bytes: 124
      Average live cells per slice (last five minutes): NaN
      Maximum live cells per slice (last five minutes): 0
      Average tombstones per slice (last five minutes): NaN
      Maximum tombstones per slice (last five minutes): 0
      Dropped Mutations: 0
      Droppable tombstone ratio: 0.00000
-----
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> |
```

Figura 3.5 — Detalle de tablestats: 100,000 escrituras locales (Write Count) sobre reservas_por_id, confirmando la carga masiva.

4. Modelo de datos (keyspace aerolinea)

El modelo lógico se diseñó a partir de las consultas de negocio (Query-Driven Modeling). A continuación, las tablas especializadas usadas por el manual y su propósito:

Tabla	Consulta que responde
asientos_contadores_por_vuelo	Q1 — Disponibilidad de asientos por clase (tabla de contadores, counter)
historial_por_pasajero	Q2 — Historial cronológico de reservas de un pasajero por rango de fechas
manifiesto_por_vuelo	Q3 — Manifiesto de vuelo ordenado por asiento, con clase, reserva y pago
ocupacion_por_ruta_mes	Q4 — % de ocupación por ruta y rango de fechas (bucketing ruta+mes)
top_vuelos_por_ingresos	Q5 — Top N de vuelos por ingresos generados en un periodo
reservas_por_id	Tabla base de reservas por identificador (soporta reglas de negocio, LWT/TTL)

Todas las tablas se consultan por su clave de partición (y clustering key cuando aplica), sin ALLOW FILTERING y sin JOINS, cumpliendo el requisito de denormalización controlada del enunciado.

5. Carga masiva de datos

La carga de datos (mínimo 100,000 reservas) se realiza con scripts de Python ubicados en la carpeta python/ del proyecto, usando Batch Writes para optimizar la inserción.

5.1 Reglas de negocio: bloqueo optimista (LWT) y expiración (TTL)

El script reglas_negocio.py incluye demostraciones de:

- Regla 2 — un asiento no puede quedar en manos de dos pasajeros (Lightweight Transactions, IF NOT EXISTS).
- TTL — una reserva pendiente caduca automáticamente si no se confirma a tiempo (en la demo, 8 segundos; en producción, 900 s = 15 min).

```
python python/reglas_negocio.py --demo-lwt --demo-ttl
```

Ejemplo de salida de la demostración de LWT: el Pasajero A confirma la reserva del asiento; el Pasajero B intenta tomar el mismo asiento y es rechazado porque ya fue tomado — solo una reserva prospera.

```

root@5376f7b73087:/app# python python/reglas_negocio.py --demo-lwt --demo-ttl
Regla 2: un asiento no puede quedar en manos de dos pasajeros

Asiento de prueba: GT0122-20261021 fila 99 letra Z -> disponible

Pasajero A intenta tomar el asiento (reserva 808df1fa-cf7e-41b3-ab7a-fd5bf85b5ac0)
-> CONFIRMADO
Pasajero B intenta tomar el MISMO asiento (reserva 8016df53-9032-4a9e-9a35-78ca05409d68)
-> RECHAZADO (alguien mas lo tomo)

Estado final del asiento: ocupado, reserva_id = 808df1fa-cf7e-41b3-ab7a-fd5bf85b5ac0
OK: solo una de las dos reservas tuvo exito; el asiento quedo con la reserva de A.

TTL: una reserva pendiente que caduca si no se confirma a tiempo

(TTL de demostracion: 8 segundos; en produccion serian 900 = 15 min)

Traceback (most recent call last):
  File "/app/python/reglas_negocio.py", line 166, in <module>
    main()
  File "/app/python/reglas_negocio.py", line 160, in main
    demo_ttl(s, codigo)
  File "/app/python/reglas_negocio.py", line 114, in demo_ttl
    session.execute(
  File "cassandra/cluster.py", line 2642, in cassandra.cluster.Session.execute
  File "cassandra/cluster.py", line 2685, in cassandra.cluster.Session.execute_async
  File "cassandra/cluster.py", line 3036, in cassandra.cluster.Session._create_response_future
  File "cassandra/query.py", line 859, in cassandra.query.bind_params
TypeError: not enough arguments for format string

```

Figura 5.1 — Salida de reglas_negocio.py: la Regla 2 (un asiento no puede quedar en manos de dos pasajeros) se cumple mediante LWT.

5.2 Verificación de la carga

Para confirmar que los datos se cargaron y distribuyeron correctamente:

```

docker exec -it cass1 cqlsh
cqlsh> USE aerolinea;
cqlsh:aerolinea> SELECT pasajero_id FROM historial_por_pasajero LIMIT 1;

```

```

PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker cp .\cql\03_consultas.cql cass1:/tmp/cql/03_consultas.cql
Successfully copied 3.72kB (transferred 5.63kB) to cass1:/tmp/cql/03_consultas.cql
PS C:\Users\Alex\Documents\Bases de Datos 2\Proyecto 1> docker exec -it cass1 cqlsh

ATTENTION: All commands will be saved to history file: /root/.cassandra/cqlsh_history
This may include sensitive information such as passwords.
To disable history, use --disable-history or set 'disabled = true' in the [history] section of cqlshrc.
See https://cassandra.apache.org/doc/latest/tools/cqlsh.html for more information.

Connected to AerolineaCluster at 127.0.0.1:9042
[cqlsh 6.1.0 | Cassandra 4.1.12 | CQL spec 3.4.6 | Native protocol v5]
Use HELP for help.
cqlsh> USE aerolinea;
cqlsh:aerolinea> SELECT pasajero_id FROM historial_por_pasajero LIMIT 1;

pasajero_id
-----
44590c5d-8844-4c23-b78a-2d22850234b7

(1 rows)
cqlsh:aerolinea>

```

Figura 5.2 — Copia del script CQL al contenedor y verificación desde cqlsh: la consulta devuelve un pasajero_id válido.

6. Consultas CQL de negocio (Q1–Q5)

El script python/consultas.py ejecuta las 5 consultas requeridas contra un vuelo y ruta de ejemplo, repitiendo cada consulta 100 veces para reportar latencia promedio, p50, p95 y máxima, al Consistency Level indicado.

```

docker exec -it <contenedor> bash
python python/consultas.py
# Usa por defecto Consistency Level QUORUM;
# ver dentro del script el parámetro para cambiarlo

```

6.1 Resumen de las 5 consultas

#	Consulta	Parámetros de ejemplo
Q1	Disponibilidad por clase	codigo_vuelo = 'GT0476-20261204'
Q2	Historial de pasajero	pasajero_id, rango de fecha_salida
Q3	Manifiesto de vuelo	codigo_vuelo = 'GT0476-20261204'
Q4	% ocupación ruta/fechas	ruta = 'MIA-LAX', mes = '2026-12'
Q5	Top N ingresos	periodo = '2026-10', LIMIT 5

```
root@5376f7b73887:/app# python python/consultas.py
Consistency Level: QUORUM | repeticiones: 100 | vuelo: GT0122-20261021 | ruta: GUA-MEX

| Consulta | filas | promedio (ms) | p50 (ms) | p95 (ms) | max (ms) |
|---|---|---|---|---|
| Q1 Disponibilidad por clase | 3 | 3.20 | 3.17 | 4.14 | 6.96 |
| Q2 Historial de pasajero | 7 | 4.16 | 4.09 | 5.04 | 7.64 |
| Q3 Manifiesto de vuelo | 146 | 4.13 | 3.94 | 5.83 | 10.77 |
| Q4 % ocupacion ruta/fechas | 3 | 3.15 | 3.05 | 4.08 | 8.05 |
| Q5 Top N ingresos | 5 | 8.13 | 8.03 | 9.72 | 16.26 |

--- Resultados de ejemplo (primeras filas) ---

Q1 Disponibilidad por clase
('economica', 29, 115)
('ejecutiva', 5, 19)
('primera', 0, 12)

Q2 Historial de pasajero
(datetime.datetime(2026, 10, 4, 21, 30), 'GT0391-20261004', 'JFK', 'MIA', 2, 'C', 'primera', 'confirmada', Decimal('1302.47'), 'pagado')
(datetime.datetime(2026, 12, 26, 12, 45), 'GT0731-20261226', 'SJO', 'GUA', 2, 'A', 'primera', 'confirmada', Decimal('1623.14'), 'pagado')
(datetime.datetime(2026, 12, 30, 13, 0), 'GT0287-20261230', 'BOG', 'GUA', 1, 'B', 'primera', 'confirmada', Decimal('2899.23'), 'pagado')
(datetime.datetime(2026, 12, 31, 13, 30), 'GT0502-20261231', 'MIA', 'MAD', 18, 'E', 'economica', 'confirmada', Decimal('303.76'), 'pagado')
(datetime.datetime(2027, 1, 7, 10, 15), 'GT0184-20270107', 'MEX', 'JFK', 21, 'E', 'economica', 'confirmada', Decimal('187.44'), 'pagado')

Q3 Manifiesto de vuelo
(1, 'A', 'Ramón del Rivero', '1971399378410', 'primera', 'confirmada', 'pagado')
(1, 'B', 'Ricardo de Barros', '6962775459510', 'primera', 'pendiente', 'pendiente')
(1, 'C', 'Loreto Cid Cordero', '4971510542875', 'primera', 'confirmada', 'pagado')
(1, 'D', 'Emma Bravo-Tejada', '6536265199598', 'primera', 'confirmada', 'pagado')
(1, 'E', 'Berto Guardia Sandoval', '3572129666363', 'primera', 'confirmada', 'pagado')

Q4 % ocupacion ruta/fechas
('GT0474-20261012', 120, 78, 65.0)
('GT0122-20261021', 180, 131, 72.78)
('GT0361-20261028', 120, 91, 75.83)

Q5 Top N ingresos
('GT0331-20261104', 'BOG-SAL', 58983.84)
('GT0686-20261027', 'MEX-SJO', 56961.48)
('GT0658-20261231', 'LAX-GUA', 56892.59)
('GT0539-20261025', 'MIA-MEX', 55699.38)
('GT0462-20261003', 'MEX-SAL', 55628.09)
root@5376f7b73887:/app#
```

Figura 6.1 — Corrida de python/consultas.py: tabla resumen de latencias de Q1–Q5 y primeras filas de resultado de cada consulta.

6.2 Q1 — Disponibilidad de asientos por clase

Usa la tabla de contadores asientos_contadores_por_vuelo, que se actualiza en cada reserva/cancelación en lugar de contar filas en tiempo de consulta.

```
SELECT clase, disponibles, ocupados
FROM asientos_contadores_por_vuelo
WHERE codigo_vuelo = 'GT0476-20261204';
```

Resultado de ejemplo:

clase	disponibles	ocupados
economica	14	70
ejecutiva	3	21
primera	4	8

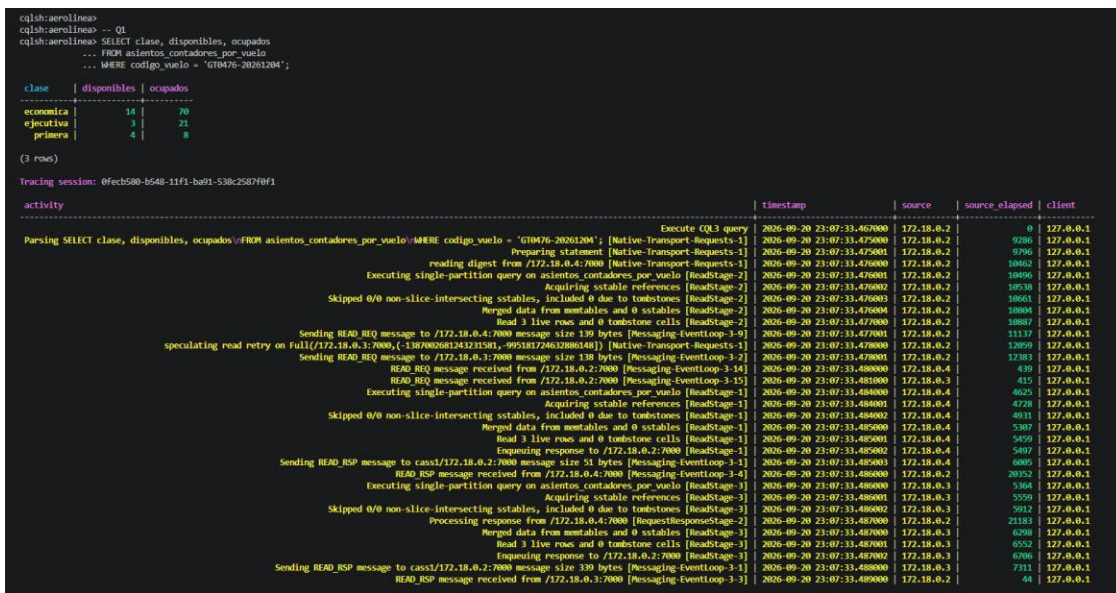


Figura 6.2 — cqlsh con TRACING ON para Q1: la traza confirma una single-partition query sobre asientos_contadores_por_vuelo (sin ALLOW FILTERING).

6.3 Q2 — Historial cronológico de un pasajero

```
SELECT fecha_salida, codigo_vuelo, origen, destino, fila, letra, clase,
       estado_reserva, monto, estado_pago
FROM historial_por_pasajero
WHERE pasajero_id = <uuid>
   AND fecha_salida >= '2026-10-01 00:00:00+0000'
   AND fecha_salida <= '2027-03-31 23:59:59+0000'
ORDER BY fecha_salida ASC;
```

Devuelve, por ejemplo, 7 filas ordenadas cronológicamente con vuelo, ruta, clase, estado de la reserva, monto pagado y estado del pago — sin necesidad de JOIN con la tabla de pagos, porque los datos están denormalizados.

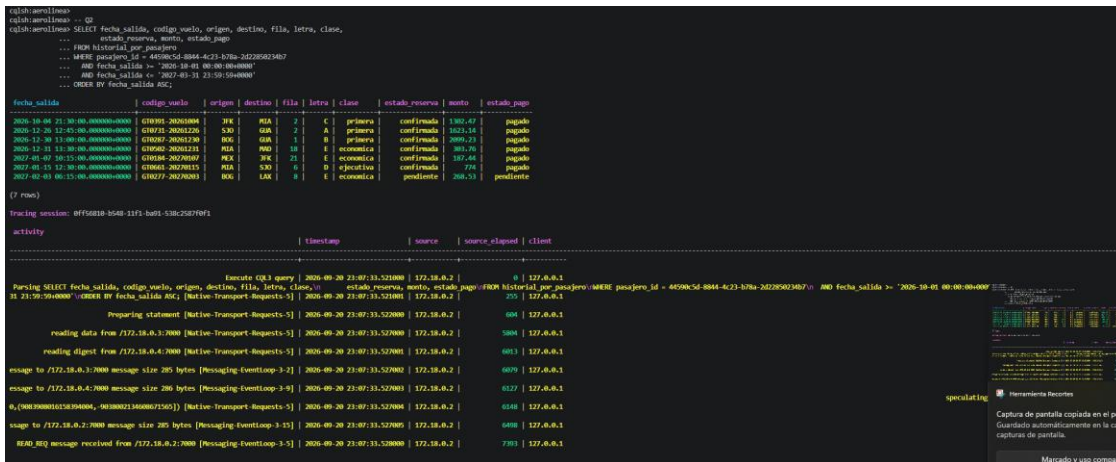


Figura 6.3 — cqlsh con TRACING ON para Q2: las 7 filas del historial del pasajero, y la traza mostrando la lectura sobre historial_por_pasajero.

6.4 Q3 — Manifiesto de vuelo ordenado y enriquecido

```
SELECT fila, letra, nombre_pasajero, identificacion, clase,
       estado_reserva, estado_pago
FROM manifiesto_por_vuelo
WHERE codigo_vuelo = 'GT0476-20261204';
```

Devuelve el listado completo de pasajeros de ese vuelo, ordenado por fila/letra de asiento, con clase, estado de reserva (confirmada/pendiente) y estado de pago (pagado/pendiente) en una sola consulta.

```
cqlsh:aerolineas> -- Q3
cqlsh:aerolineas> SELECT fila, letra, nombre_pasajero, identificacion, clase,
... estado_reserva, estado_pago
... FROM manifiesto_por_vuelo
... WHERE codigo_vuelo = 'GT0476-20261204';
```

fila	letra	nombre_pasajero	identificacion	clase	estado_reserva	estado_pago
1	A	Lola Sáenz Díez	7220497964654	primera	confirmada	pagado
1	B	Lidia Manzanares Palomares	4194416885634	primera	confirmada	pagado
1	D	Severo Mascará Guillén	0319360537455	primera	confirmada	pagado
1	E	Luz Guzmán Caro	5283186774330	primera	confirmada	pagado
1	F	Benita Castellanos-Castell	4059550714167	primera	confirmada	pagado
2	A	Felipa Ferrando Gimeno	8443685595630	primera	pendiente	pendiente
2	D	Fabio Ballester Espada	3015764867706	primera	confirmada	pagado
2	F	Coral Valcárcel Barrio	7506546361458	primera	pendiente	pendiente
3	B	Manola Cazorla Pablo	5241555118219	ejecutiva	confirmada	pagado
3	C	Blas Cuéllar Salcedo	6649609076113	ejecutiva	pendiente	pendiente
3	D	Jordi Cáceres Vilanova	4059476955038	ejecutiva	confirmada	pagado
3	E	Quique Maza-Chico	3748949144108	ejecutiva	confirmada	pagado
3	F	Pascuala Rocamora Coll	8883291396141	ejecutiva	confirmada	pagado
4	B	Guadalupe Luque Garay	1388932989677	ejecutiva	confirmada	pagado
4	C	Antonio Puente	1147636853950	ejecutiva	confirmada	pagado
4	D	Jorge Mendoza Bueno	6094494464741	ejecutiva	pendiente	pendiente
4	E	Jacinto Castilla Aragón	4024932110809	ejecutiva	confirmada	pagado
4	F	Sebastián Pardo	6883200759533	ejecutiva	confirmada	pagado
5	A	Augusto Capdevila Llano	7146354246934	ejecutiva	confirmada	pagado
5	B	Leoncio Ropero Amorós	3797669500228	ejecutiva	confirmada	pagado
5	C	Isidoro Collado Leiva	2999815297214	ejecutiva	confirmada	pagado
5	D	Gaspar Montalbán-Rico	4672130848316	ejecutiva	confirmada	pagado
5	F	Pánfilo Palacios Huertas	8839398605771	ejecutiva	confirmada	pagado
6	A	Mireia Carlos Saura	7080414908854	ejecutiva	confirmada	pagado
6	B	Calixto del Vilaplana	0909940824386	ejecutiva	confirmada	pagado
6	C	Baudelio Bermúdez Palacio	8222583143093	ejecutiva	confirmada	pagado
6	D	Maricela Aller Acedo	6600095328518	ejecutiva	confirmada	pagado
6	E	Cayetana Morata Benítez	9733590767842	ejecutiva	confirmada	pagado
6	F	Nydia Uría Tolosa	8494666205818	ejecutiva	confirmada	pagado
7	A	Rita Bernal Moliner	5898541727839	economica	confirmada	pagado
7	B	Palmira Santamaría Montaña	5441146359284	economica	confirmada	pagado
7	C	María Manuela Murillo Agullo	8915671611932	economica	confirmada	pagado
7	D	Lucho Murillo Pedrero	0391623781775	economica	confirmada	pagado
7	E	Maricruz Lasa	0084697693241	economica	confirmada	pagado
8	A	Tomasa Olimpia Garmendia Enríquez	7309988774847	economica	confirmada	pagado
8	B	Fausto Martín Alberto Uribe	6282470977148	economica	confirmada	pagado
8	E	Lara de Pereira	5760408978682	economica	confirmada	pagado
8	F	Kike Alonso Teruel	6794558997219	economica	confirmada	pagado
9	A	Leoncio Álvaro-Arroyo	7764038337270	economica	confirmada	pagado
9	B	Teo Ramírez Sabater	7525126873884	economica	confirmada	pagado

Figura 6.4 — Manifiesto del vuelo GT0476-20261204 en cqlsh, ordenado por fila y letra de asiento, con clase, estado de reserva y estado de pago.

6.5 Q4 — % de ocupación por ruta y rango de fechas

```
SELECT fecha_salida, codigo_vuelo, capacidad, confirmadas
FROM ocupacion_por_ruta_mes
WHERE ruta = 'MIA-LAX' AND mes = '2026-12'
AND fecha_salida >= '2026-12-01 00:00:00+0000'
AND fecha_salida <= '2026-12-31 23:59:59+0000';
```

Ejemplo de salida: vuelo GT0476-20261204, capacidad 120, 82 reservas confirmadas → 68.3% de ocupación aproximada. El bucketing por ruta+mes evita particiones gigantes.

```
cqlsh:aerolineas> -- Q4
cqlsh:aerolineas> SELECT fecha_salida, codigo_vuelo, capacidad, confirmadas
... FROM ocupacion_por_ruta_mes
... WHERE ruta = 'MIA-LAX' AND mes = '2026-12'
... AND fecha_salida >= '2026-12-01 00:00:00+0000'
... AND fecha_salida <= '2026-12-31 23:59:59+0000';
```

fecha_salida	codigo_vuelo	capacidad	confirmadas
2026-12-04 13:45:00+0000:000000	GT0476-20261204	120	82

(1 row)

Tracing session: 1094250-9540-11f1-b0f1-50b2507f9f1

activity	source	source_elapsed	client	timestamp
Execute CQL query	127.0.0.2	0	127.0.0.1	2026-09-20 23:07:33
Parsing SELECT fecha_salida, codigo_vuelo, capacidad, confirmadas FROM ocupacion_por_ruta_mes WHERE ruta = 'MIA-LAX' AND mes = '2026-12' AND fecha_salida >= '2026-12-01 00:00:00+0000' AND fecha_salida <= '2026-12-31 23:59:59+0000'	127.0.0.2	427	127.0.0.1	2026-09-20 23:07:33
Preparing statement	127.0.0.2	725	127.0.0.1	2026-09-20 23:07:33
reading data from /127.0.0.1:7000 [Native-Transport-Requests-6]	127.0.0.2	2305	127.0.0.1	2026-09-20 23:07:33
reading digest from /127.0.0.1:7000 [Native-Transport-Requests-6]	127.0.0.2	2063	127.0.0.1	2026-09-20 23:07:33
Sending HEAD_REQ message to /127.0.0.1:7000 message size 144 bytes [Messaging-EventLoop-3-2]	127.0.0.2	2342	127.0.0.1	2026-09-20 23:07:33
Sending HEAD_REQ message to /127.0.0.1:7000 message size 144 bytes [Messaging-EventLoop-3-2]	127.0.0.2	2000	127.0.0.1	2026-09-20 23:07:33
HEAD_REQ message received from /127.0.0.1:7000 [Messaging-EventLoop-3-15]	127.0.0.3	34	127.0.0.1	2026-09-20 23:07:33
HEAD_REQ message received from /127.0.0.1:7000 [Messaging-EventLoop-3-14]	127.0.0.4	31	127.0.0.1	2026-09-20 23:07:33
Executing single-partition query on ocupacion_por_ruta_mes [ReadStage-1]	127.0.0.3	403	127.0.0.1	2026-09-20 23:07:33
Executing single-partition query on ocupacion_por_ruta_mes [ReadStage-5]	127.0.0.4	253	127.0.0.1	2026-09-20 23:07:33
Acquiring stable references [ReadStage-1]	127.0.0.3	208	127.0.0.1	2026-09-20 23:07:33
Acquiring stable references [ReadStage-5]	127.0.0.4	500	127.0.0.1	2026-09-20 23:07:33
Skipped 0/0 non-slice-intersecting satables, included 0 due to tombstones [ReadStage-5]	127.0.0.4	415	127.0.0.1	2026-09-20 23:07:33

Figura 6.5 — cqlsh con TRACING ON para Q4: consulta sobre ocupacion_por_ruta_mes para la ruta MIA-LAX en diciembre 2026.

6.6 Q5 — Top N de vuelos por ingresos generados

```
SELECT codigo_vuelo, ruta, total_centavos
FROM top_vuelos_por_ingresos
WHERE periodo = '2026-10'
LIMIT 5;
```

La tabla mantiene el orden DESC por monto mediante la clustering key, por lo que el LIMIT 5 se resuelve directamente en el servidor, sin ordenar ni filtrar en el cliente. Ejemplo de resultado (top 5 de octubre 2026): MEX-SJO, MIA-MEX, MEX-SAL, JFK-LAX y MIA-MAD, con ingresos entre ~Q54,218 y ~Q56,961 (montos en centavos en la tabla).

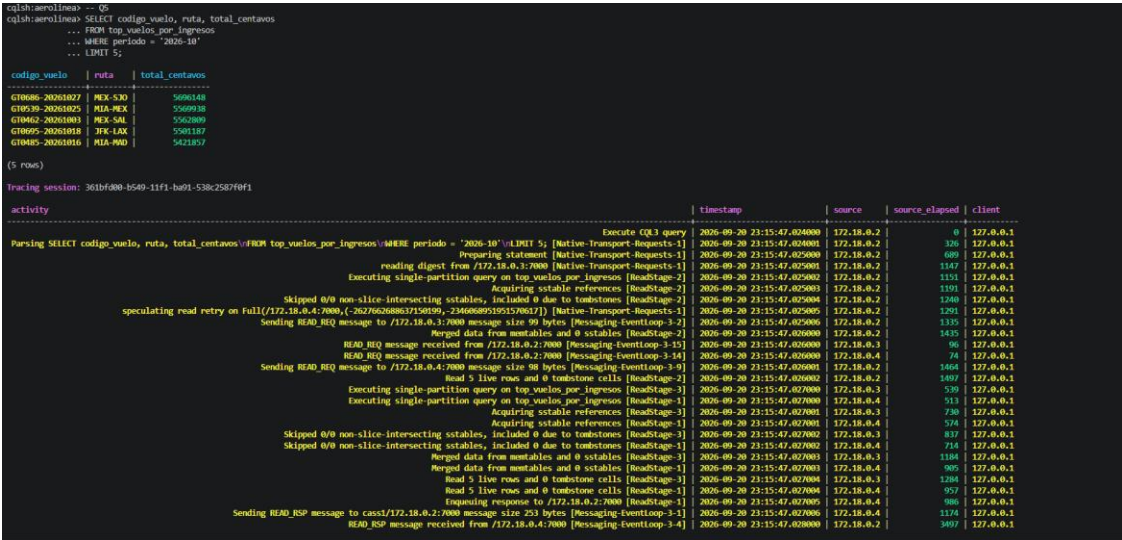


Figura 6.6 — cqlsh con TRACING ON para Q5: Top 5 vuelos por ingresos de octubre 2026, leído directamente en orden DESC desde top_vuelos_por_ingresos.

6.7 Latencias observadas (QUORUM, 100 repeticiones)

Consulta	Filas	Prom. (ms)	p50 (ms)	p95 (ms)	max (ms)
Q1 Disponibilidad por clase	3	1.82 – 3.20	1.58 – 3.17	2.77 – 4.14	3.55 – 6.96
Q2 Historial de pasajero	7	2.06 – 4.16	1.92 – 4.09	2.84 – 5.04	5.82 – 7.64
Q3 Manifiesto de vuelo	146	3.21 – 4.13	3.17 – 3.94	3.85 – 5.83	4.23 – 10.77
Q4 % ocupación ruta/fechas	3	1.92 – 3.15	1.89 – 3.05	2.35 – 4.08	3.02 – 8.05
Q5 Top N ingresos	5	5.88 – 8.13	5.46 – 8.03	8.71 – 9.72	15.36 – 16.26

Los rangos reflejan varias corridas del script; las latencias más altas ocurrieron cuando el clúster acababa de reiniciarse tras una caída simulada de nodo. En general, todas las consultas resuelven en pocos milisegundos y ninguna requiere ALLOW FILTERING.

```

root@171c27e71b9a:/app# python python/consultas.py
Consistency Level: QUORUM | repeticiones: 100 | vuelo: GT0122-20261021 | ruta: GUA-MEX

| Consulta | filas | promedio (ms) | p50 (ms) | p95 (ms) | max (ms) |
|---|---|---|---|---|---|
| Q1 Disponibilidad por clase | 3 | 1.83 | 1.58 | 3.83 | 4.62 |
| Q2 Historial de pasajero | 7 | 2.06 | 1.92 | 2.84 | 5.98 |
| Q3 Manifiesto de vuelo | 146 | 3.63 | 3.46 | 4.94 | 5.82 |
| Q4 % ocupacion ruta/fechas | 3 | 2.27 | 2.06 | 3.72 | 3.98 |
| Q5 Top N ingresos | 5 | 7.01 | 6.89 | 8.71 | 16.29 |

--- Resultados de ejemplo (primeras filas) ---

Q1 Disponibilidad por clase
('economica', 29, 115)
('ejecutiva', 5, 19)
('primera', 0, 12)

Q2 Historial de pasajero
(datetime.datetime(2026, 10, 4, 21, 30), 'GT0391-20261004', 'JFK', 'MIA', 2, 'C', 'primera', 'confirmada', Decimal('1302.47'), 'pagado')
(datetime.datetime(2026, 12, 26, 12, 45), 'GT0731-20261226', 'SJO', 'GUA', 2, 'A', 'primera', 'confirmada', Decimal('1623.14'), 'pagado')
(datetime.datetime(2026, 12, 30, 13, 0), 'GT0287-20261230', 'BOG', 'GUA', 1, 'B', 'primera', 'confirmada', Decimal('2099.23'), 'pagado')
(datetime.datetime(2026, 12, 31, 13, 30), 'GT0502-20261231', 'MIA', 'MAD', 18, 'E', 'economica', 'confirmada', Decimal('303.76'), 'pagado')
(datetime.datetime(2027, 1, 7, 10, 15), 'GT0184-20270107', 'MEX', 'JFK', 21, 'E', 'economica', 'confirmada', Decimal('187.44'), 'pagado')

Q3 Manifiesto de vuelo
(1, 'A', 'Ramón del Rivero', '1971399378410', 'primera', 'confirmada', 'pagado')
(1, 'B', 'Ricardo de Barros', '6962775459510', 'primera', 'pendiente', 'pendiente')
(1, 'C', 'Loreto Cid Cordero', '4971519542875', 'primera', 'confirmada', 'pagado')
(1, 'D', 'Emma Bravo-Tejada', '6536265199598', 'primera', 'confirmada', 'pagado')
(1, 'E', 'Berto Guardia Sandoval', '3572129666363', 'primera', 'confirmada', 'pagado')

Q4 % ocupacion ruta/fechas
('GT0474-20261012', 120, 78, 65.0)
('GT0122-20261021', 180, 131, 72.78)
('GT0361-20261028', 120, 91, 75.83)

Q5 Top N ingresos
('GT0331-20261104', 'BOG-SAL', 58983.84)
('GT0686-20261027', 'MEX-SJO', 56961.48)
('GT0658-20261231', 'LAX-GUA', 56892.59)
('GT0539-20261025', 'MIA-MEX', 55699.38)
('GT0462-20261003', 'MEX-SAL', 55628.09)
root@171c27e71b9a:/app#

```

Figura 6.7 — Segunda corrida de consultas.py (clúster ya estabilizado): latencias ligeramente distintas pero mismo orden de magnitud.

```

root@171c27e71b9a:/app# python python/consultas.py
Consistency Level: QUORUM | repeticiones: 100 | vuelo: GT0122-20261021 | ruta: GUA-MEX

| Consulta | filas | promedio (ms) | p50 (ms) | p95 (ms) | max (ms) |
|---|---|---|---|---|---|
| Q1 Disponibilidad por clase | 3 | 1.82 | 1.59 | 2.77 | 3.55 |
| Q2 Historial de pasajero | 7 | 2.64 | 2.75 | 3.26 | 3.41 |
| Q3 Manifiesto de vuelo | 146 | 3.21 | 3.17 | 3.85 | 4.23 |
| Q4 % ocupacion ruta/fechas | 3 | 1.92 | 1.89 | 2.35 | 3.02 |
| Q5 Top N ingresos | 5 | 5.88 | 5.46 | 8.80 | 15.36 |

--- Resultados de ejemplo (primeras filas) ---

Q1 Disponibilidad por clase
('economica', 29, 115)
('ejecutiva', 5, 19)
('primera', 0, 12)

Q2 Historial de pasajero
(datetime.datetime(2026, 10, 4, 21, 30), 'GT0391-20261004', 'JFK', 'MIA', 2, 'C', 'primera', 'confirmada', Decimal('1302.47'), 'pagado')
(datetime.datetime(2026, 12, 26, 12, 45), 'GT0731-20261226', 'SJO', 'GUA', 2, 'A', 'primera', 'confirmada', Decimal('1623.14'), 'pagado')
(datetime.datetime(2026, 12, 30, 13, 0), 'GT0287-20261230', 'BOG', 'GUA', 1, 'B', 'primera', 'confirmada', Decimal('2099.23'), 'pagado')
(datetime.datetime(2026, 12, 31, 13, 30), 'GT0502-20261231', 'MIA', 'MAD', 18, 'E', 'economica', 'confirmada', Decimal('303.76'), 'pagado')
(datetime.datetime(2027, 1, 7, 10, 15), 'GT0184-20270107', 'MEX', 'JFK', 21, 'E', 'economica', 'confirmada', Decimal('187.44'), 'pagado')

Q3 Manifiesto de vuelo
(1, 'A', 'Ramón del Rivero', '1971399378410', 'primera', 'confirmada', 'pagado')
(1, 'B', 'Ricardo de Barros', '6962775459510', 'primera', 'pendiente', 'pendiente')
(1, 'C', 'Loreto Cid Cordero', '4971519542875', 'primera', 'confirmada', 'pagado')
(1, 'D', 'Emma Bravo-Tejada', '6536265199598', 'primera', 'confirmada', 'pagado')
(1, 'E', 'Berto Guardia Sandoval', '3572129666363', 'primera', 'confirmada', 'pagado')

Q4 % ocupacion ruta/fechas
('GT0474-20261012', 120, 78, 65.0)
('GT0122-20261021', 180, 131, 72.78)
('GT0361-20261028', 120, 91, 75.83)

Q5 Top N ingresos
('GT0331-20261104', 'BOG-SAL', 58983.84)
('GT0686-20261027', 'MEX-SJO', 56961.48)
('GT0658-20261231', 'LAX-GUA', 56892.59)
('GT0539-20261025', 'MIA-MEX', 55699.38)
('GT0462-20261003', 'MEX-SAL', 55628.09)
root@171c27e71b9a:/app#

```

Figura 6.8 — Tercera corrida de consultas.py, confirmando la consistencia de los resultados entre ejecuciones.

6.9 Verificación con tracing en cqlsh

Para inspeccionar el plan de ejecución de cada consulta puede activarse el tracing dentro de cqlsh:

```
docker exec -it cass1 cqlsh
cqlsh:aerolinea> TRACING ON;
cqlsh:aerolinea> SELECT ... ; -- cualquiera de las consultas Q1-Q5
```

El tracing confirma que cada consulta ejecuta como single-partition query (partición única) sobre la tabla correspondiente — es decir, resuelve por clave de partición, sin escaneo completo ni ALLOW FILTERING.

7. Pruebas de tolerancia a fallos y Consistency Level

El script `python/pruebas_consistencia.py` ejecuta 100 operaciones de lectura y 100 de escritura por cada Consistency Level (ONE, QUORUM, ALL), para tres escenarios de disponibilidad del clúster.

```
python python/pruebas_consistencia.py --escenario 3_nodos
python python/pruebas_consistencia.py --escenario 1_nodo_caido
python python/pruebas_consistencia.py --escenario 2_nodos_caidos
python python/pruebas_consistencia.py --resumen # tabla comparativa final
```

Antes de cada escenario con nodos caídos se debe detener manualmente el/los contenedor(es) correspondientes (`docker stop cass2`, `docker stop cass3`) y luego reiniciarlos (`docker start cass2 cass3`) antes del siguiente escenario, como se describe en la sección 3.2. Los resultados se guardan automáticamente en `resultados/consistencia.csv`.

7.1 Resultados por escenario

Escenario	CL	Lectura	Escritura	Observación
3 nodos (sin caídas)	ONE	100/100	100/100	Funciona normalmente
3 nodos (sin caídas)	QUORUM	100/100	100/100	Funciona normalmente
3 nodos (sin caídas)	ALL	100/100	100/100	Funciona normalmente
1 nodo caído (cass3)	ONE	100/100	100/100	Sin impacto
1 nodo caído (cass3)	QUORUM	100/100	100/100	Sin impacto (2 de 3 réplicas alcanzables)
1 nodo caído (cass3)	ALL	0/100	0/100	Unavailable=100 (requiere las 3 réplicas)
2 nodos caídos (cass2, cass3)	ONE	100/100	100/100	Sin impacto (1 réplica basta)
2 nodos caídos (cass2, cass3)	QUORUM	0/100	0/100	Unavailable=100 (no alcanza el quórum)
2 nodos caídos (cass2, cass3)	ALL	0/100	0/100	Unavailable=100

```
root@171c27e71b9a:/app# C
root@171c27e71b9a:/app# python python/pruebas_consistencia.py --escenario 3_nodos
Escenario: 3_nodos | 100 operaciones por CL y tipo

ONE   lectura  exitos 100/100 avg 1.81 ms p95 2.59 ms
ONE   escritura exitos 100/100 avg 1.17 ms p95 1.46 ms
QUORUM lectura  exitos 100/100 avg 2.82 ms p95 3.80 ms
QUORUM escritura exitos 100/100 avg 1.09 ms p95 1.46 ms
ALL   lectura  exitos 100/100 avg 2.75 ms p95 4.21 ms
ALL   escritura exitos 100/100 avg 1.30 ms p95 1.78 ms

Guardado en resultados/consistencia.csv. Siguiendo escenario, o --resumen al terminar.
root@171c27e71b9a:/app#
```

Figura 7.1 — Escenario 3_nodos: ONE, QUORUM y ALL responden 100/100 en lectura y escritura.

```
root@171c27e71b9a:/app# python python/pruebas_consistencia.py --escenario 1_nodo_caido
(aviso) cass3 no resuelve: se asume caido
Escenario: 1_nodo_caido | 100 operaciones por CL y tipo

ONE  lectura  exitos 100/100 avg 1.59 ms p95 2.22 ms
ONE  escritura exitos 100/100 avg 1.05 ms p95 1.81 ms
QUORUM lectura  exitos 100/100 avg 2.58 ms p95 3.52 ms
QUORUM escritura exitos 100/100 avg 1.47 ms p95 2.09 ms
ALL   lectura  exitos 0/100 avg - ms p95 - ms Unavailable=100
ALL   escritura exitos 0/100 avg - ms p95 - ms Unavailable=100

Guardado en resultados/consistencia.csv. Siguiendo escenario, o --resumen al terminar.
root@171c27e71b9a:/app#
```

Figura 7.2 — Escenario 1_nodo_caido: ONE y QUORUM funcionan con éxito; ALL falla con Unavailable=100.

```
Guardado en resultados/consistencia.csv. Siguiendo escenario, o --resumen al terminar.
root@171c27e71b9a:/app# python python/pruebas_consistencia.py --escenario 2_nodos_caidos
(aviso) cass2 no resuelve: se asume caido
(aviso) cass3 no resuelve: se asume caido
Escenario: 2_nodos_caidos | 100 operaciones por CL y tipo

ONE  lectura  exitos 100/100 avg 1.42 ms p95 2.05 ms
ONE  escritura exitos 100/100 avg 0.85 ms p95 1.22 ms
QUORUM lectura  exitos 0/100 avg - ms p95 - ms Unavailable=100
QUORUM escritura exitos 0/100 avg - ms p95 - ms Unavailable=100
ALL   lectura  exitos 0/100 avg - ms p95 - ms Unavailable=100
ALL   escritura exitos 0/100 avg - ms p95 - ms Unavailable=100

Guardado en resultados/consistencia.csv. Siguiendo escenario, o --resumen al terminar.
█
```

Figura 7.3 — Escenario 2_nodos_caidos: solo ONE funciona; QUORUM y ALL fallan con Unavailable=100.

7.2 Latencias por Consistency Level (escenario 3 nodos)

CL	Operación	Éxitos	Prom. (ms)	p50 (ms)	p95 (ms)
ONE	lectura	100/100	1.81	1.74	2.59
ONE	escritura	100/100	1.17	1.13	1.46
QUORUM	lectura	100/100	2.82	2.77	3.80
QUORUM	escritura	100/100	1.09	1.03	1.46
ALL	lectura	100/100	2.75	2.48	4.21
ALL	escritura	100/100	1.30	1.24	1.78

Conclusión: con los 3 nodos activos, ONE es el más rápido pero ofrece la consistencia más débil; QUORUM es el mejor equilibrio entre rendimiento y confiabilidad (tolera 1 nodo caído); ALL exige que todas las réplicas respondan y por eso es el primero en fallar (Unavailable) apenas cae un nodo, aunque su latencia con el clúster completo es similar a QUORUM.

```

| Escenario | CL escritura | Escrituras exitosas | Filas leídas con ALL | Estado |
|---|---|---|---|---|
| 1_nodo_caído | ALL | 0 | 0 | OK |
| 1_nodo_caído | ONE | 100 | 100 | OK |
| 1_nodo_caído | QUORUM | 100 | 100 | OK |
| 2_nodos_caídos | ALL | 0 | 0 | OK |
| 2_nodos_caídos | ONE | 100 | 100 | OK |
| 2_nodos_caídos | QUORUM | 0 | 0 | OK |
| 3_nodos | ALL | 100 | 100 | OK |
| 3_nodos | ONE | 100 | 100 | OK |
| 3_nodos | QUORUM | 100 | 100 | OK |
root@171c27e71b9a:/app# python python/pruebas_consistencia.py --resumen
| Escenario | CL | Operacion | Exitos | % éxito | Prom (ms) | p50 (ms) | p95 (ms) | Errores |
|---|---|---|---|---|---|---|---|---|
| 3_nodos | ONE | lectura | 100/100 | 100% | 1.81 | 1.74 | 2.59 | - |
| 3_nodos | ONE | escritura | 100/100 | 100% | 1.17 | 1.13 | 1.46 | - |
| 3_nodos | QUORUM | lectura | 100/100 | 100% | 2.82 | 2.77 | 3.80 | - |
| 3_nodos | QUORUM | escritura | 100/100 | 100% | 1.09 | 1.03 | 1.46 | - |
| 3_nodos | ALL | lectura | 100/100 | 100% | 2.75 | 2.48 | 4.21 | - |
| 3_nodos | ALL | escritura | 100/100 | 100% | 1.30 | 1.24 | 1.78 | - |
| 1_nodo_caído | ONE | lectura | 100/100 | 100% | 1.59 | 1.51 | 2.22 | - |
| 1_nodo_caído | ONE | escritura | 100/100 | 100% | 1.05 | 0.95 | 1.81 | - |
| 1_nodo_caído | QUORUM | lectura | 100/100 | 100% | 2.58 | 2.46 | 3.52 | - |
| 1_nodo_caído | QUORUM | escritura | 100/100 | 100% | 1.47 | 1.44 | 2.09 | - |
| 1_nodo_caído | ALL | lectura | 0/100 | 0% | - | - | - | Unavailable=100 |
| 1_nodo_caído | ALL | escritura | 0/100 | 0% | - | - | - | Unavailable=100 |
| 2_nodos_caídos | ONE | lectura | 100/100 | 100% | 1.42 | 1.34 | 2.05 | - |
| 2_nodos_caídos | ONE | escritura | 100/100 | 100% | 0.85 | 0.70 | 1.22 | - |
| 2_nodos_caídos | QUORUM | lectura | 0/100 | 0% | - | - | - | Unavailable=100 |
| 2_nodos_caídos | QUORUM | escritura | 0/100 | 0% | - | - | - | Unavailable=100 |
| 2_nodos_caídos | ALL | lectura | 0/100 | 0% | - | - | - | Unavailable=100 |
| 2_nodos_caídos | ALL | escritura | 0/100 | 0% | - | - | - | Unavailable=100 |
root@171c27e71b9a:/app#

```

Figura 7.4 — Salida de `pruebas_consistencia.py --resumen`: tabla comparativa de disponibilidad y latencias por escenario y Consistency Level.

7.3 Verificación manual de un nodo caído

```

docker stop cass2
docker exec cass1 nodetool status
# cass2 -> DN (Down)

docker stop cass3
docker exec cass1 nodetool status
# cass2 y cass3 -> DN (Down); solo cass1 permanece UN

```

8. Solución de problemas comunes

Problema	Causa probable / Solución
Unavailable=100 en QUORUM o ALL	Uno o más nodos están detenidos. Verificar con <code>nodetool status</code> y reiniciar los nodos con <code>docker start</code> .
<code>TypeError: not enough arguments for format string</code>	El script de reglas de negocio recibió menos parámetros de los que la consulta parametrizada (<code>%s</code>) espera; revisar la lista de valores que se pasa a <code>session.execute()</code> .
"(aviso) cassN no resuelve: se asume caído" al iniciar <code>pruebas_consistencia.py</code>	Mensaje informativo: el script detectó que ese nodo no responde y ajusta el escenario automáticamente; no es un error si coincide con el escenario que se está probando.
<code>cqlsh</code> no conecta	Confirmar que el contenedor está arriba (<code>docker exec -it cass1 cqlsh</code>) y que se usa el nombre de host correcto dentro de la red de Docker.
Las consultas tardan más de lo normal tras una caída	Es esperado justo después de un <code>nodetool repair</code> o de reiniciar un nodo, mientras el clúster reconstruye el estado; las latencias se normalizan en corridas posteriores.

9. Referencia rápida de comandos

```
# Clúster
docker compose up -d
docker exec cass1 nodetool status
docker stop <nodo>           docker start <nodo>
docker exec cass1 nodetool repair aerolinea
docker exec cass1 nodetool tablestats aerolinea.<tabla>

# Datos y reglas de negocio
python python/reglas_negocio.py --demo-lwt --demo-ttl

# Consultas de negocio (Q1-Q5)
python python/consultas.py

# Pruebas de Consistency Level y tolerancia a fallos
python python/pruebas_consistencia.py --escenario 3_nodos
python python/pruebas_consistencia.py --escenario 1_nodo_caido
python python/pruebas_consistencia.py --escenario 2_nodos_caidos
python python/pruebas_consistencia.py --resumen

# cqlsh
docker exec -it cass1 cqlsh
USE aerolinea;
TRACING ON;
```